A Modular Task-to-Device Integration Framework for Industrial Agents

***Abstract*—Industrial agents must turn task goals into device operations across changing production contexts and heterogeneous interfaces, while preserving evidence of both commands and process responses. The platform connects registered agent harnesses and direct-operation calls to a shared dispatcher. Scene mappings link semantic controls to configured drivers; command receipts remain separate from later process observations. When approval is required, the handler checks that the line, product, recipe, and active run still match the approved context. In simulation, one registered harness completed six sequential tasks across two scenes, each with three admitted candidate settings, synthetic approval, a verified trial receipt, and a run-tagged sample. All six single post-trial probes fell below their target bands. A mock-driver test rejected injected stale contexts before writes. Four simulated deterministic direct-node missions reported final primary values within their target bands without model-selected setpoints. These results cover selected configurations; repeatability, comparative optimization performance, human-review effectiveness, and physical-plant performance remain unverified.**

***Index Terms*—Industrial agents, industrial communication, industrial informatics, production systems, semantic data integration.**

I. Introduction

INDUSTRIAL-AGENT deployments must connect a changing process context to heterogeneous equipment without giving a language model unrestricted access to control points. A line-speed proposal, for example, depends on the active product, recipe, run, engineering limits, and node that represents the control. It must also reach the correct device through a configured protocol, and its receipt must not be confused with a later process response. When these elements are implemented separately for each agent and production scene, integrating a new engine or line requires repeating substantial interface and context work.

Prior work has advanced industrial-agent supervision, language-model control and programmable logic controller (PLC) code generation, modular manufacturing skills, and digital-twin and shop-floor integration [1]–[11]. Related platforms ground approved actions in Asset Administration Shell (AAS) or process and instrumentation diagram (P&ID) models, and integrate sensor, manufacturing execution system (MES), and enterprise resource planning (ERP) evidence streams [12]–[14].

The deployment problem addressed here is how to map distinct industrial scenes to a common operation interface, provide active production context to agent tasks, and translate a proposal into a bounded, traceable device operation. We present a configuration-driven integration architecture, not a new control law, optimization algorithm, or experimentally validated multi-agent controller. Its central design choice is to keep harness execution, scene-specific signal and driver mappings, and device-write governance modular while connecting them through named industrial operations.

The contributions are threefold:

1) A scene-integration model binds production context (line–product–recipe–run), semantic acquisition and control nodes, protocol drivers, objectives, and engineering guards. Distinct processes can therefore be represented through configuration and a common operation interface, while protocol-specific access remains in drivers.
2) An integration runtime registers replaceable harness adapters against a common task-operation interface, assembles current production context at each turn, and scopes available operations to channel policy and resource bindings. Multiple engines and task roles are supported by the architecture; the experiments evaluate a narrower set of routes and configurations.
3) A governed write path combines shared dispatch checks with route-specific admission, conditional human approval, post-approval context revalidation, and device-side receipt grading. Owning services retain command receipts separately from later process observations.

Fig. 2 presents the architecture and its module boundaries; Fig. 1 follows one task proposal through operation dispatch, device access, and evidence capture. Evaluation examines configured protocol and scene integration, model-backed task-to-device operations on a thermal twin, and separate route, approval, and process checks. These evidence units use different configurations and are not pooled as repeated trials. They verify selected software paths; they do not validate all agent–scene combinations or establish multi-agent performance or physical-plant behavior.

Section II reviews related work. Section III presents the system architecture and execution flow. Section IV explains agent context, scene integration, and governed device access. Section V reports the protocol, onboarding, agent-task, approval, and process-mission evidence. Sections VI and VII discuss the findings and conclude the paper.

II. Related Work

Table I summarizes selected studies that connect agent or software-agent functions to industrial services, equipment, or production data and report an implementation or evaluation. The selection locates the integration problem addressed here; it is neither exhaustive nor a matched comparison of system performance.

A. Industrial Agents and Plant Integration

Industrial-agent studies span process supervision, control generation, and equipment-facing assistance. LLM frame-

works address end-to-end automation and industrial cyber-physical-system control [1], [2]; related work generates PLC code or combines human guidance with control [6], [7]. Other studies examine resource capabilities, shop-floor coordination, and digital-twin fault handling [8]–[10], as well as natural-language machine access and robotic-line control [4], [11]. Biskupovic et al. are especially close in using agents to supervise industrial systems: their agents select services from a component library using performance and computational-load metrics, monitor online performance, and reconfigure service selection when performance degrades [5]. Taken together, this strand studies agent behavior at the level of control, service choice, or equipment interaction; the question here is how those agents connect to scene-specific signals and governed operations.

Xia et al. connect cognitive, digital-twin, and physical-automation layers and organize twin information as state snapshots, event messages, and plan sequences [15]. Other nearby designs expose factory-module operations through the Model Context Protocol (MCP) with state-based tool filtering, constrain manufacturing plans, or compose semantic recipes from OPC Unified Architecture (OPC UA) skills [3], [16], [17]. Several recent systems also combine agent execution with approval, industrial data, or audit records. Graul et al. ground proposals in the Asset Administration Shell (AAS) and route approved actions through OPC UA Methods, with signed approvals and audit logging [12]. Schall separates advisory language-model agents from process and instrumentation diagram (P&ID)-grounded verification and execution, and evaluates structural checks across five simulated process scenarios [13]; its context includes supervisory control and data acquisition (SCADA) and standard operating procedure (SOP) information. Lin and Chen build a governed sensor–MES–ERP digital thread for closed-loop monitoring and control on real manufacturing records [14]. These systems address complementary parts of industrial-agent deployment, including process reasoning, service adaptation, asset grounding, action validation, and evidence organization. This paper addresses a related but distinct integration boundary: how to connect replaceable agent harnesses and task roles to different configured production scenes while preserving line–product–recipe–run context, semantic signal mappings, and a governed parameter-write path. Whereas Biskupovic et al. study agent-based service selection and reconfiguration, this architecture centers on scene configuration and a shared operation boundary that resolves configured device nodes and retains command receipts separately from process observations. The contribution is an integration architecture and implementation, not a new process-control law. Table I compares reported mechanisms and evaluation settings only; it does not rank features or performance.

The operation checks complement safety-aware decision support [18] and operate alongside the plant's configured process protection.

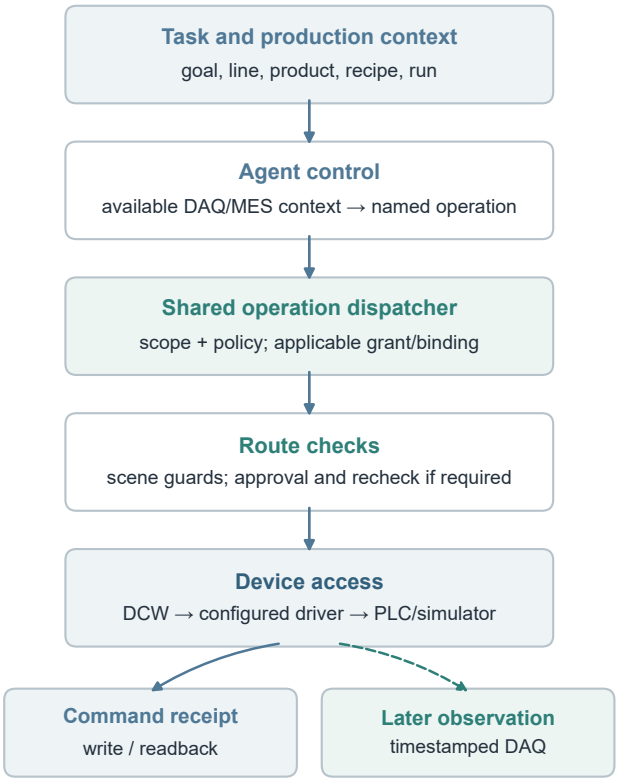


Fig. 1. Task execution path from context to device and observation. The agent uses available acquisition and MES context to propose a named operation. Shared dispatch checks precede route guards and any required approval and context revalidation. Configured drivers provide device access; command receipts and later process observations are retained separately.

B. Production Context, Agent Interfaces, and Automation

Industrial integration also depends on abstractions that are not specific to agent architectures. The Model Context Protocol (MCP) specifies interoperable tool discovery and call mechanisms [19]. The platform dispatcher's operation-level scope, policy, grant, and binding checks are distinct from MCP transport authorization. In parallel, ISA-88 and ISA-95 describe recipes and production operations [20], [21]. Run-to-run methods connect process observations to successive production runs [22]. Human–automation research also formalizes the allocation of information acquisition, analysis, decisions, and actions between people and automation [23]. At the agent level, AgentSpec and CaMeL address tool behavior and capability-based information flow [24], [25]. These standards provide interfaces for production context, tool interaction, and information flow. This work connects them to scene configuration and execution-time checks; the evaluation exercises selected links under simulator and mock-driver conditions.

III. System Design

The architecture connects two configurable sides of an industrial deployment: an agent collaboration runtime and a production-scene configuration. The runtime supplies task context and scoped operations; scene configuration maps process semantics to acquisition and control nodes, drivers, targets, and guards. Named operations connect these sides to shared dispatch and governed device access. The design separates task execution from operation dispatch and keeps command receipts distinct from measured process responses.

TABLE I
REPORTED OPERATION MECHANISMS AND EVALUATIONS IN SELECTED PLANT-FACING STUDIES. EVALUATION SETTINGS AND OUTCOMES ARE SOURCE-SPECIFIC; THIS TABLE DOES NOT COMPARE PERFORMANCE ACROSS STUDIES.

System	Reported operation or integration mechanism	Reported evaluation
Xia et al. [15]	Cognitive, twin, and physical layers; twin snapshots, event messages, and plan sequences	Robotic automation and process-simulation cases
Biskupovic et al. [5]	Agent-based selection and online reselection of services using performance and computational-load metrics	Reported adaptability and performance improvement in service-oriented industrial cyber-physical systems
Köhle et al. [3]	Injected live state and state-based filtering; MCP-to-OPC UA operations and Message Queuing Telemetry Transport (MQTT) coordination	Nine simulated production challenges
Vieira da Silva et al. [16]	Constrained multi-step plans over MCP manufacturing functions	Laboratory multi-step process
Köhle et al. [17]	Semantic recipes composed from OPC UA skills; virtual commissioning	Unity assembly-workstation proof of concept
Graul et al. [12]	AAS-grounded proposals, signed human approval, PLC-validated OPC UA Methods, and immutable audit log	Two laboratory PoCs; single scripted runs
Lin and Chen [14]	Governed sensor–MES–ERP digital thread with SOP-, risk-, and capability-aware action gating	25,275 real records; operational metrics
Schall [13]	P&ID-, SCADA-, and SOP-grounded proposal verification with gated execution	Five simulated scenarios; 43 proposals and 50 LLM runs
This work	Harness-independent task-operation interface; scene-bound context and DAQ/DCW mappings; bounded named operations; separate command and observation records	Five-protocol simulator check; model-backed single-agent channel; harness tasks; approval and route checks; four exploratory single-trajectory missions

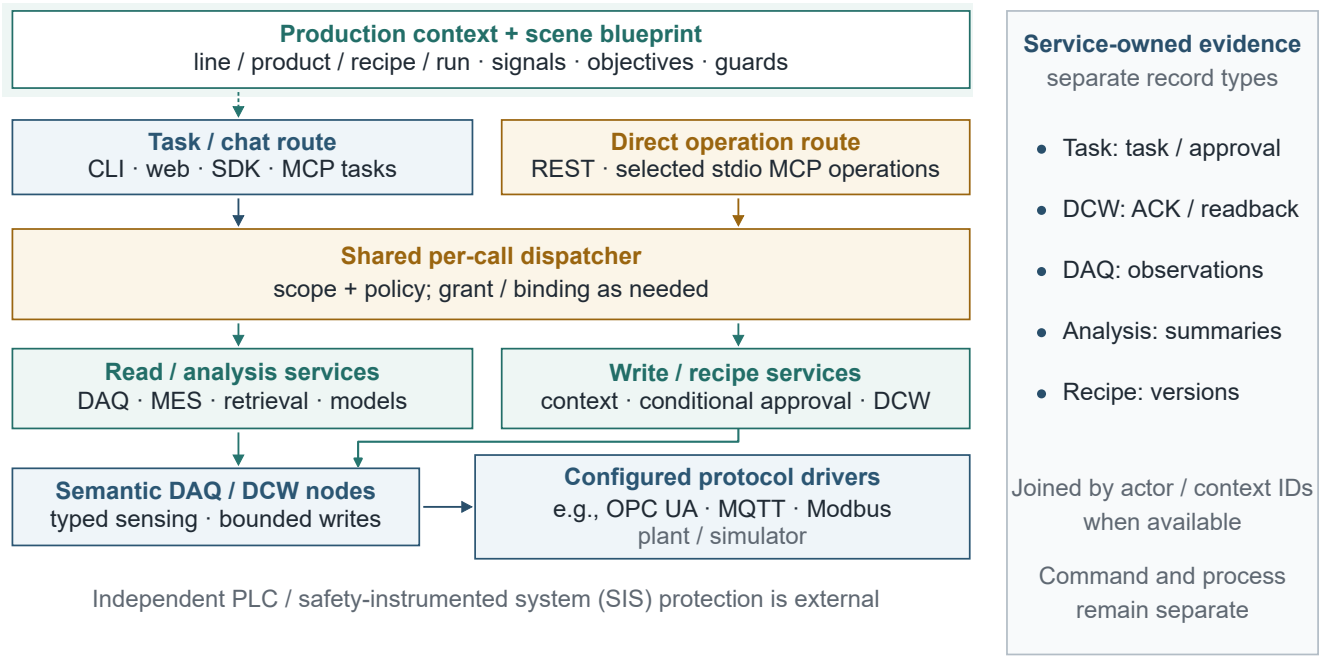


Fig. 2. Architecture and record ownership. Task/chat routes enter the collaboration runtime; direct REST and stdio-MCP calls enter shared dispatch checks without creating a task. One dashed link marks configuration. Write routes apply interval admission and, when configured, approval; recipe dispatch omits the previous-recipe window. The shared dispatcher always checks scope and channel policy, with grant and resource-binding checks applied when required by the operation. Owning services retain distinct records; plant controllers and safety systems retain equipment interlocks.

A. Design Rationale and Requirements

Five requirements shape the design: scene variability, harness variability, operation authority, outcome observability, and extensibility. Scene configuration binds production records and semantic DAQ/DCW nodes to drivers, objectives, and process guards. Harness adapters isolate agent-engine interfaces from named operations. Shared dispatch checks and route-specific write limits govern authority, while service-owned task, receipt, and observation records preserve outcomes. Registered interfaces allow additional agent engines

and device drivers to be integrated without changing the operation contract. Tool visibility alone does not authorize a write; all actuator writes pass through the DCW admission point.

Commissioning associates the line with products, recipes, active-run rules, signal meanings, units, driver mappings, and admissible limits. At runtime, the context builder assembles the active line–product–recipe–run state and relevant process summaries for each agent turn. An operation handler resolves live state again before dispatch, so the prompt context informs a proposal but does not substitute for current authorization or admission checks. Direct operations resolve the current context at invocation without creating a task.

B. Layered Architecture

Fig. 2 summarizes the access routes, shared dispatch path, production services, semantic I/O, configured drivers, and service ownership of records. Task and chat routes enter the collaboration runtime, which contains workspaces, channels, task queues, shared memory, mailboxes, task templates, and harness adapters. A *task route* is an execution path through this runtime; a *harness adapter* connects an agent engine to its task interface. Multiple registered agents can participate through shared tasks and channel context, with each agent's available tools constrained by its binding and channel policy. Direct REST tool calls and the stdio MCP operation bridge bypass task execution in that band (the upper-right direct-operation entry in Fig. 2) and enter the shared governance block without creating a task or starting a harness turn; MCP management and task methods remain on their service routes. Direct calls resolve current route state at invocation, without an agent-turn prompt or a task record. The governance block projects the operation catalog and applies the per-call checks. Write handlers apply admission limits and request approval when the operation and its configuration require it. The context block supplies the active line–product–recipe–run tuple to the gate and scene maps, objectives, and guards to the handlers. The dashed link from that block in Fig. 2 marks this configuration relationship; the runtime assembles prompt-time context from the same records. The service band, behind a route-handler interface that selects an adapter, provides acquisition, write control, analysis, retrieval, and optional model services. Field I/O resolves semantic nodes to built-in or plugin drivers for a plant or simulator. Services retain task, command, observation, and configuration records separately, linked to actor and production context.

C. Cross-Module Execution Flow

Fig. 3 summarizes a task turn from entry through operation execution, with route-specific evidence retained by owning services. Registered harness adapters receive a scoped tool catalog during setup and supported registry refreshes; the runtime assembles prompt-time production context from current records for each agent turn. For each returned tool call, the shared dispatcher rechecks invocation scope, channel policy, and applicable grant and binding conditions before handler selection. The selected handler then applies route-specific guards, active-run checks, approval revalidation, and device-I/O steps as applicable. Command receipts and later process observations remain distinct records.

For an industrial operation u by agent a in channel c, define the shared dispatch condition as

$$\mathcal{A}(a, c, u) = \bigwedge_{X \in \mathcal{C}(u)} X(a, c, u), \tag{1}$$

where S, P, G, and B denote invocation scope, current channel policy, applicable industrial grant, and the agent's binding to referenced resources, respectively. $\mathcal{C}(u) \subseteq \{S, P, G, B\}$ is the set of predicates declared for operation u; it always contains S and P. The grant predicate G is included when the resolved channel, target line, and owner/grant policy require it; explicit exemptions omit it. The binding predicate B is included when the operation declares resource references. A declared no-reference operation omits B. Each included check must pass before handler selection. The policy predicate includes channel tool selection, profile, and duty-separation checks where configured. Once these predicates pass, the selected route applies its guards, active-run checks, admission limits, and any required approval. When approval is required, the handler revalidates its context before device dispatch.

For direct REST invocation, the service establishes a and c before applying these checks: an agent token must match the requested agent identifier, or an authenticated user must be the channel owner or an administrator. The manager loads the channel-agent record and rechecks scope and policy; applicable grant and binding checks follow. Handlers resolve live production state from request identifiers and owning services.

Fig. 4 gives reference pseudocode summarizing the registered-harness task path. Direct REST tool invocation bypasses the task runtime and harness but enters the same shared dispatcher. Route-specific timing, approval, and evidence rules are detailed in Section IV.

IV. Module Functions and Integration

Section III defines the architecture and shared dispatch contract. This section specifies the runtime context passed to adapters and the route-specific admission, approval, and evidence steps.

A. Task, Harness, and Context

Task and chat entry points create or join a workspace channel. The task runtime manages membership, roles, task state, and shared execution context; a harness registry adapts agent engines to a common execution interface. This separation lets agents collaborate through shared tasks and channel resources without coupling their engine-specific turn protocol to a production scene. At each turn, the prompt builder assembles the task, active scene description, line–product–recipe–run state, channel profile, agent binding, and retrieved memory. The resulting context supports process-aware proposals, while the dispatcher independently checks authority and the operation handler resolves current state before a write.

Direct REST and selected stdio-MCP operation calls reach the same dispatcher without a task or harness turn. REST

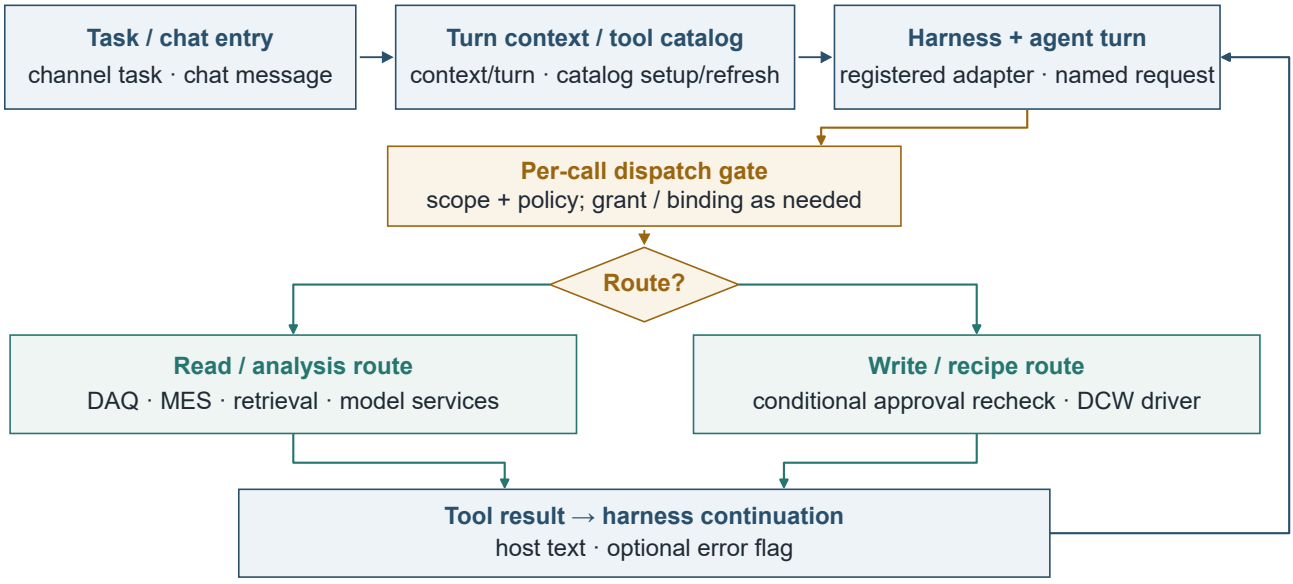


Fig. 3. Registered-harness task flow. The runtime invokes the adapter with a scoped catalog. Scope and channel policy are checked for every call; grant applies to non-exempt industrial operations, and binding applies when resource references are declared. Failed checks return tool errors. The read path retrieves data. The write path applies route guards, conditional approval and its context revalidation, DCW admission, and receipt grading; a later process observation is separate. Results return to the harness. Device-level interlocks remain with the configured plant-control layer.

Require: task q, channel c, bound agent a, registered scoped catalog T
Ensure: task result; any retained evidence remains with its owning service
$x_t \leftarrow \textsc{ResolvePromptContext}(c, q)$
$m \leftarrow \textsc{RetrieveMemory}(q)$
$p \leftarrow \textsc{AssemblePrompt}(q, x_t, m)$
$h \leftarrow \textsc{InvokeRegisteredHarness}(a, p, T)$
for all tool request u returned by h **do**
  $g \leftarrow \textsc{CheckScopeAndChannelPolicy}(a, c, u)$
  if not g **then**
    $\textsc{EmitToolErrorToHarness}$(h,u); **continue**
  end if
  if u is an industrial operation **then**
    $v \leftarrow \textsc{CheckApplicableGrantBinding}(a, u)$
    if not v **then**
      $\textsc{EmitToolErrorToHarness}$(h,u)
      continue
    end if
  end if
  $k \leftarrow \textsc{SelectHandler}(u)$
  $r \leftarrow \textsc{ExecuteRouteHandler}(k, a, c, u)$ ▷ handler resolves live state and may persist route-specific records
  $\textsc{EmitDeclaredToolResultToHarness}$(h,r)
end for
return task state and tool results

Fig. 4. Reference pseudocode for the registered-harness task path. Direct operation calls use the shared dispatcher from their own entry point. Each route persists records through its owning service and returns its declared response to its caller; task-route results return to the harness.

authentication and identity resolution follow Section III-C; each handler reads live route state at invocation. Handlers receive the resolved actor, channel, and operation arguments through the same interface for either entry route; records are keyed by the identifiers supplied in each request. A registered task additionally receives prompt context and a scoped tool catalog. Catalog visibility alone does not grant write authority: the dispatcher checks authority at invocation.

B. Shared Dispatch and Device Admission

Each operation must satisfy the predicates in Eq. (1) before handler selection; the selected handler then revalidates route-specific state. Scene configuration maps semantic signals and controls to drivers and specifies process objectives and guards. Capability profiles and tool policies narrow available operations, while resource bindings constrain accessible nodes, controls, and recipes. Optional DAQ, MES, analysis, retrieval, optimization, and digital-twin services are invoked through registered handlers. The application supplies its optimization strategy; the framework coordinates context, operation access, and execution.

The target band describes a desired process outcome. It is not the admissible write interval: the latter is computed from engineering, product, recipe, and route-specific limits below. A candidate can therefore satisfy all write constraints yet fail to move the measured process value into its target band.

Actuator writes converge on the shared DCW admission point. For an online node write, the admissible value interval is

$$\mathcal{I}_n^{\text{online}}(t) = \mathcal{H}_n \cap \mathcal{L}_n^{\text{base}} \cap \mathcal{L}_n^{\text{prod}}(t) \cap \mathcal{L}_n^{\text{rec}}(t), \qquad (2)$$

where $\mathcal{H}_n$ is the hard engineering range and $\mathcal{L}_n^{\text{base}}$, $\mathcal{L}_n^{\text{prod}}$, and $\mathcal{L}_n^{\text{rec}}$ are the baseline, active-product, and active-recipe limits;

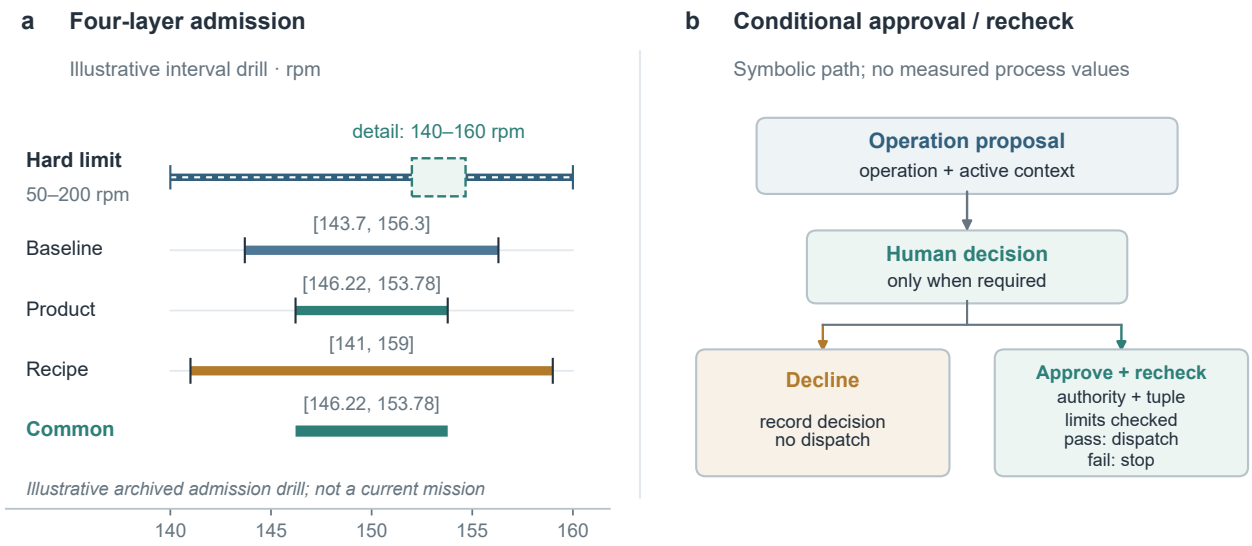


Fig. 5. Admission and approval boundaries. (a) Illustrative rotational-speed limits from a separate archived drill. The hard-limit track spans 50–200 rpm; the lower axis expands its 140–160 rpm segment. Online writes use the intersection in Eq. (2); recipe dispatch omits the previous recipe window. (b) When approval is required, a decline prevents dispatch; approval is followed by authority, context, and limit rechecks.

the $\mathcal{L}$ limits here are deliberately named apart from the policy and binding predicates of (1). Missing optional limits impose no restriction; an empty intersection rejects the request and identifies the excluding layer. Recipe dispatch validates the replacement recipe against the hard, baseline, and product limits rather than the previous recipe window. Step, interval, and hold-window checks apply as configured for each route. These application-level checks bound commands alongside independent programmable-logic-controller and safety-instrumented protection. They do not replace equipment interlocks or certify the safety of a plant deployment.

C. Approval, Execution, and Evidence

Routes add timing checks and, when required by operation and configuration, approval checks. Outside benchmark mode, online agent and manual node writes enforce a 60 s minimum interval between successful writes. Recipe trials use a per-line cadence (300 s by default), and recipe intervals are anchored to approval. Manual approval binds an actor and proposed operation to candidate values, applicable limits, expiry, and the active line–product–recipe–run tuple. After approval, the handler rechecks authority and compares the current tuple with the approved tuple before entering DCW dispatch. Single-parameter waits expire after 180 s by default; recipe-dispatch approvals use a 30-minute window. A changed recipe definition requires a new `recipe_apply` proposal.

Fig. 5(a) illustrates the four-layer interval in Eq. (2) using rotational-speed limits from a separate archived admission drill. Panel (b) shows the decision and rechecks when approval is required.

A `recipe_trial` illustrates the operation sequence: the handler checks the bound recipe, active run, candidate parameters, and route timing; if required, it requests approval; after revalidation, it submits a bounded batch through the configured driver and returns a per-parameter receipt. The caller can obtain later DAQ observations separately and may then update, apply, or roll back a recipe. A trial neither saves nor applies a recipe version. The driver registry maps semantic controls to configured protocol drivers, keeping protocol details out of agent operation arguments.

For a concrete illustration, consider a simulated coating line whose semantic line-speed control is mapped to a Modbus TCP write node. A request to change the setpoint from 37 to 39 m/min is inside a 28–48 m/min recipe window; the remaining applicable limits and timing checks must also pass. If the configured policy requires approval, the proposal binds the candidate and active production context, and the handler rechecks that context before dispatch. The driver then returns a graded receipt. A later acquisition sample is stored separately and can be associated with the operation when the route provides the necessary context and timestamps. This example illustrates the handoffs, not a recorded human decision or a particular process response; the simulator onboarding replay in Section V used test-only automatic approval.

Receipts and observations answer different questions. A command result is graded as readback-verified, transport-acknowledged, or unverified; a readback mismatch can trigger up to three reads at 2 s intervals. Recipe batches retain per-parameter verification outcomes. A readback-verified result records that the returned value matched the request; a transport acknowledgement alone does not establish that the device applied it, and an unverified result does not establish a matching readback. DAQ stores subsequent timestamped process observations, while recipe history, approvals, and task records remain with their owning services. Identifiers and timestamps can link these records when available, but a later observation alone does not prove that the command caused the measured change. Section V evaluates selected parts of this evidence chain under the stated test conditions.

V. Experimental Evaluation

The evaluation examines selected parts of the task-to-device integration path using local simulators and a mock driver. A registered-harness test examines task execution and its retained operation records; Fig. 6 shows the associated process probes. Other checks examine protocol and node access, direct-operation records, and approval-context rejection. Separately, four deterministic direct-node missions illustrate how requests and observations are recorded across different process scenes. They cover injection molding, biaxially oriented polyethylene terephthalate (BOPET) film production, continuous annealing, and anaerobic/anoxic/oxic wastewater treatment. Table III summarizes their primary outcomes; their trajectories appear in Fig. S2 of the supplementary evidence bundle. The missions and harness tasks address different evaluation questions and use different setups; neither is a control or repetition of the other.

A. Test Design and Evidence Rules

All process experiments use local simulators, not physical equipment. The four missions use deterministic scenario-specific routines, a fixed random seed, and the direct-node interface; no language model selects their setpoints. In the available benchmark source, an automatic keep judgment follows a write and precedes the next observation, except in BOPET, where it follows the response. Annealing and wastewater logs also record later reversion decisions. These are software events, not human pre-approval. The full executed runtime snapshot was not retained, so the records cannot be replayed as a complete trace.

Each mission contributes one trajectory, with its initial observation as a within-trajectory reference rather than an independent control. A requested setpoint is an action, not a device readback; subsequent process values are observations. The primary outcome is the mission-reported final value relative to its target band. Intermediate observations are not independent repetitions.

B. Protocol and Node Integration

The protocol test configured simulator endpoints using Modbus TCP, Modbus RTU, OPC UA, MQTT, and HTTP; its contemporaneous report records successful connectivity for each. In a Modbus TCP check, seven samples were collected and the simulated process reached within 3 °C of a requested 182 °C setpoint after 24 s. In a separate fault case, a 185 °C write was read back while the process value remained frozen at 150 °C; monitoring raised an alarm within 2 s. A reported link-fault exercise detected loss of connection and recovered after simulator restart. The complete per-sample protocol trace was not retained, so these results are limited to the contemporaneous summary.

The BOPET line had nine devices and 49 signals: 30 writable setpoints and 19 measurements. Its four requests addressed three control nodes. A separate onboarding replay configured one simulated Modbus TCP device with four write and seven acquisition nodes, together with scalar, vector, table, and image forms from a manufacturing execution system (MES) simulator. All seven acquisition nodes yielded samples, four invalid writes were rejected, and four legal writes were confirmed by readback; all 23 verifier checks passed. A test-only automatic approval allowed a 37-to-39 m/min line-speed change, confirmed by the simulator. This was one integration instance. The source record does not fully identify the build, and the MES image used a local storage fallback.

C. Registered-Harness Tasks and Direct Operations

A registered task harness (an open-model protocol engine operating over stdio JSON-RPC) configured with `glm-5.3-flash` executed six sequential tasks across two simulated scenes. Each task supplied a current control read, process measurement, target band, and three candidate settings admitted by the dispatch predicates of (1) and the interval admission of (2); band attainment itself was not pre-verified. Application records associate the selected candidate with a registered-harness tool proposal; independent model-provider request and response logs were not retained. With synthetic administrator approval, all six tasks completed and each retained an approved proposal, a recipe-trial record, a 1/1 verified device receipt, and a run-tagged sample. The runner initiated a 20 s delay after each receipt/readback checkpoint before probing the driver. Fig. 6 pairs each pre-task process measurement with its single post-trial probe. In the first film task, the pre-task value was 55.82 μm, above the 48–52 μm target band, while the probe was 45.69 μm, below it. The other film probes were 46.69 and 45.69 μm; annealing probes were 679.60, 677.70, and 678.10 °C against a 700–760 °C target. Thus, despite task completion and verified receipts, all six probes were below target. The three tasks per scene shared one simulator instance, and samples used a local SQLite fallback because the isolated online queue and time-series service were unavailable. These paired checkpoints do not establish causation, convergence, independent replication, or normal online acquisition.

Separate checks without model-generated actions exercised a fixed-call registered task and a direct REST operation in each scene, using synthetic approval and isolated environments. Each returned a verified trial receipt and a run-tagged sample; the task route also retained a task–approval–receipt–sample link, but task completion was not asserted. At 20 s, film samples were both 46.37 μm; annealing samples were 674.80 and 675.20 °C, respectively, all below the corresponding target bands. The samples were explicit driver probes stored using a local database fallback with online DAQ disabled. The two checks document which records each route retained. They used separate environments and did not trace both calls to the same handler; neither adds a four-mission trajectory.

D. Approval Context and Dispatch

An independent component test exercised human-in-the-loop (HITL) approval using simulated decisions and a mock driver. A proposal binds the line, product, recipe, and active run to the approval context; after approval the handler compares that context with the current one before dispatch. The test

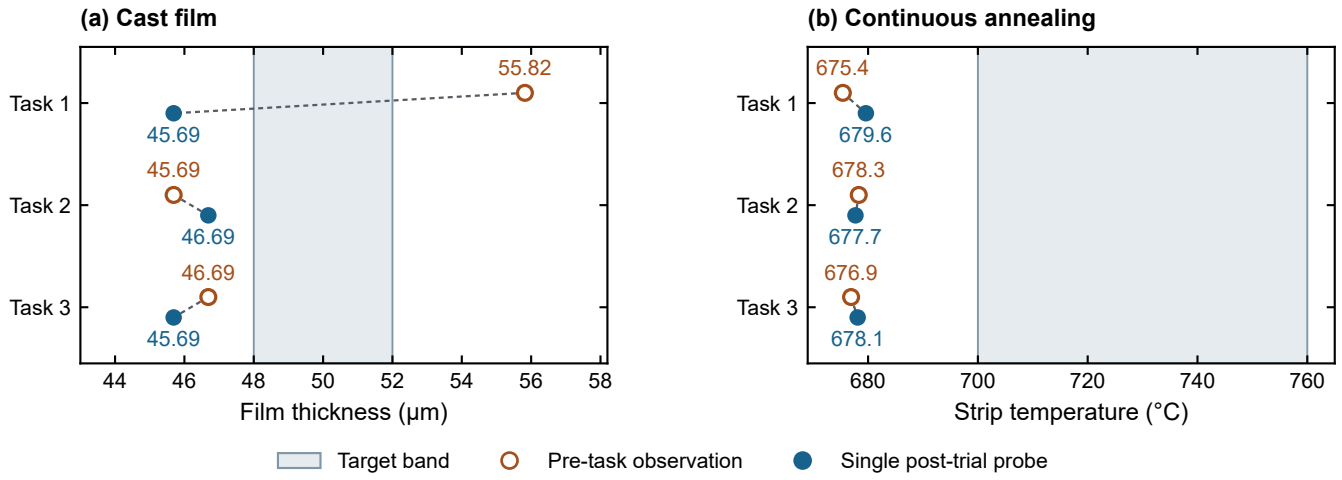


Fig. 6. Pre-task observations and single post-trial probes for six sequential registered-harness tasks in two simulated scenes. Shading denotes the target band; task numbers indicate order within each scene. Dashed connectors pair observations within a task, and vertical offsets separate markers without changing their measured values. The connectors do not represent a time series. The runner waited 20 s after receipt/readback before each probe. Tasks in each scene shared a simulator instance; synthetic approval and a local SQLite fallback were used.

TABLE II
INTEGRATION AND APPROVAL CHECKS. THE PROTOCOL, ONBOARDING, AND HARNESS CHECKS USED SIMULATORS; THE APPROVAL TEST USED MOCK DRIVERS AND SIMULATED DECISIONS.

Check	Observed outcome
Protocol access	Five protocol connections succeeded
Node onboarding	All seven acquisition nodes sampled; all invalid writes rejected; all legal writes read back
Registered harness	Six tasks completed with verified receipts; all six probes outside target
Approval context	All changed contexts refused before mock writes
Without context comparison	Six product or run changes admitted; all line or recipe changes still refused

TABLE III
PRIMARY PROCESS VALUES IN THE FOUR SIMULATED MISSIONS. EACH ROW IS ONE TRAJECTORY; THE FINAL VALUE IS THE MISSION-REPORTED VALUE.

Scenario	Target band	Initial → final
Injection	Part weight: 32.15–32.85 g	31.296 → 32.36 g
Film	Thickness: 24.3–25.7 μm	28.03 → 25.30 μm
Annealing	Hardness: 89–101 HV	134.9 → 99.5 HV
Wastewater	Dissolved oxygen: 2.8–4.0 mg/L	0.194 → 3.54 mg/L

changed each of the four fields for recipe application, trial, and rollback: all injected stale contexts were refused before mock writes. A further injected change between authorization revalidation and the final check was also refused. An unchanged approved context produced the expected mock writes, whereas a rejected proposal produced none.

To examine the contribution of the context comparison, a test-only ablation removed that comparison while retaining the other checks. Six product or run changes then reached the mock driver and each produced two writes and an execution record. The remaining six line or recipe changes were still blocked by other checks. Thus the comparison supplied the tested product/run protection in this component setup; the result is not a measurement of operator effectiveness, human decision time, physical safety, or behavior during network and device I/O. Table II summarizes the integration and approval checks. Table III reports the separate process-mission outcomes.

E. Direct-Node Missions: Injection Molding and Film

The direct-node film mission uses a different thickness target from the registered-harness tasks. The injection-molding mission targeted 32.5 ± 0.35 g. Starting at 31.296 g, the routine requested holding pressure from 45 to 57.73 bar and holding time from 6 to 10.56 s; observations rose to 31.93 and 32.09 g. A further pressure request to 62 bar preceded the final 32.36 g observation within the target band. This single sequence does not isolate the effect of each parameter.

The BOPET mission targeted 25.0 ± 0.7 μm. From an initial 28.03 μm, four requests changed casting speed, fast-roll speed, rail width, and casting speed again; successive observations were 26.48, 26.06, 25.78, and 25.30 μm. The final value was within the band, while the first three remained above it. Reported melt temperatures were 291.3–291.8 °C, within the configured 268–300 °C window. These observations describe one simulated sequence.

F. Direct-Node Missions: Annealing and Wastewater

The direct-node annealing mission measures hardness, whereas the registered-harness tasks record strip temperature (Fig. 6). The annealing mission combined a 95 ± 6 HV hardness target with a line-speed increase. From 134.9 HV, a zone-2 temperature request from 710 to 850 °C preceded a structured observation of 99.18 HV; a speed request from 140 to 152 m/min preceded 99.74 HV. A 164 m/min candidate was reverted after event text reported 101.0 HV, at the upper target boundary with no remaining hardness margin. The summary separately reported final hardness of 99.5 HV and retained speed of 152 m/min, 8.57% above the initial setting, not a

measured throughput gain. A zone-3 read failed before the zone-2 action. Final tensile-strength and surface-defect measurements were not archived, so the record cannot establish that all quality constraints were met.

The wastewater mission targeted dissolved oxygen (DO) at 3.4 ± 0.6 mg/L. Five blower-frequency increases from 26 to 34.1 Hz accompanied a rise from 0.194 to 3.567 mg/L in the structured aeration sequence. Subsequent alkali and coagulant dosing addressed pH and phosphorus; observed total phosphorus fell from 0.662 to 0.5507, 0.4607, and 0.413 mg/L. A later blower reduction was reverted after insufficient margin. The summary separately reported final DO of 3.54 mg/L and a simulator-defined cost index changing from 182.6 to 180.2, not a measured operating cost. The complete final effluent vector was not archived, so these values do not verify all discharge constraints.

G. Evidence Synthesis

The six registered-harness tasks retained operation records, and each single post-trial probe fell below its target band. Separately, the four direct-node missions reported final primary values within their respective target bands. Neither evaluation establishes repeatability, causal effects of individual actions, comparative performance, human-approval effectiveness, or transfer to a physical plant.

VI. Discussion

A. Interpretation

Protocol connectivity and the onboarding replay test different parts of device integration: communication with configured simulator endpoints and access to declared acquisition and control nodes. They do not show that arbitrary equipment can be integrated without adaptation. The entry-path checks document which records were retained for each tested call. Because the calls used separate environments and no handler-level trace was retained, they cannot establish route equivalence or confirm that both reached the same handler. In the approval-context test, stale product and run identities reached mock writes when tuple comparison was removed and were rejected when it was enabled. This isolates the comparison's contribution within that setup.

The six registered-harness tasks and four direct-node missions answer different questions. The harness tasks link approvals and receipts to single post-trial probes; the missions record one process trajectory per scenario. They use separate setups, including different film target bands and different annealing measures, so mission endpoints do not benchmark the harness probes. Across the missions, requested actions precede measured responses, including a reverted annealing candidate. In wastewater treatment, the structured aeration sequence and final mission summary also refer to different observations. These records describe trajectories, not isolated effects of individual commands.

The missions also illustrate the role of scene-specific configuration. Part weight, film thickness, hardness, and dissolved oxygen have different units, target bands, and available actuators. The operation–response structure accommodates these differences, but the trajectories do not identify which architectural component caused an outcome. Related approaches likewise connect industrial operations to semantic skills, equipment context, or process validation [12], [13], [17]. The present study complements those designs by describing a shared operation path and reporting responses in four simulated scenes.

B. Scope and Limitations

The optimization evidence comprises one deterministic simulated trajectory per scene, without independent replicates or a matched alternative controller. It cannot establish statistical repeatability, superiority to another platform, or the effect of individual controls. The missions use direct-node operations and do not test the full range of entry routes or language-model decisions described by the architecture. Final tensile-strength and surface-defect readings for annealing and the complete final wastewater effluent vector are missing, limiting conclusions about secondary constraints. The process model and simulator configuration also limit transfer to physical production.

The mission records show automated judgments, not human pre-approval. The approval-context test uses simulated decisions and mock drivers; it tests a software condition, not operator performance or the effectiveness of human review [26], [27]. Software operation restrictions are not a complete security or safety assessment [28]–[30]; independent plant protection remains necessary [31]. Future work should repeat missions across initial conditions, preserve all terminal quality measurements, compare matched controllers and entry paths, and evaluate the system with suitable physical equipment and operator oversight.

This modular separation is a property of the implemented software structure. The current evidence does not measure whether it reduces engineering effort, operating cost, or reliability; those outcomes require matched deployments.

VII. Conclusion

The platform links industrial task context to scene-specific device operations through shared dispatch, configured drivers, and separate command and observation records. The evaluation covered simulator protocol and node access, registered-harness tasks, mock-driver approval checks, and four direct-node missions. The four mission endpoints were within their primary target bands. Separately, all six single probes after harness tasks remained below their bands despite verified receipts. These simulator and mock-driver results do not establish repeatability, comparative benefit, human-review effectiveness, or performance on a physical plant.

Data Availability: The historical Supplementary File S1 contains route and integration-check records and the onboarding replay log; it does not contain the current four-mission traces. A separate supplementary evidence bundle contains pseudonymized mission logs and selected registered-harness records. The manuscript source, vector figures, and figure-source data accompany the submission package. No physical-plant data are included.

REFERENCES

[1] Y. Xia, N. Jazdi, J. Zhang, C. Shah, and M. Weyrich, "Control industrial automation system with large language model agents," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205539.

[2] K.-C. Huang, H.-T. Lin, C.-H. Lin, and M.-T. Huang, "Large language model integration in industrial cyber-physical systems: A multi-modal architecture for automated control generation," in *Proc. Int. Conf. on Control, Automation and Diagnosis (ICCAD)*, 2026, pp. 1–6, doi: 10.1109/ICCAD69956.2026.11643124.

[3] K. Köhle, D. Anicic, T. A. Runkler, and R. Graf, "LLM-driven multi-agent control for skill-based smart manufacturing," in *Proc. IEEE 31st Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2026, pp. 1–8, doi: 10.1109/ETFA68377.2026.11725257.

[4] M. Waseem, K. Bhatta, C. Li, and Q. Chang, "Pretrained LLMs as real-time controllers for robot operated serial production line," *Mach. Learn.: Eng.*, vol. 2, no. 1, 2026, art. no. 015006; doi: 10.1088/3049-4761/ae4ee8.

[5] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber–physical systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2719–2728, 2025, doi: 10.1109/TII.2024.3514178.

[6] Z. Liu *et al.*, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Trans. Softw. Eng.*, vol. 52, no. 5, pp. 1672–1687, 2026, doi: 10.1109/TSE.2026.3667895.

[7] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-LLM-in-the-loop framework for context-aware control," in *Proc. IEEE 64th Conf. on Decision and Control (CDC)*, 2025, pp. 172–179, doi: 10.1109/CDC57313.2025.11312156.

[8] J. Lim and I. Kovalenko, "A large language model-enabled control architecture for dynamic resource capability exploration in multi-agent manufacturing systems," in *Proc. IEEE 21st Int. Conf. on Automation Science and Engineering (CASE)*, 2025, pp. 2088–2095, doi: 10.1109/CASE58245.2025.11163802.

[9] Z. Zhao *et al.*, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Adv. Eng. Inform.*, vol. 69, 2026, art. no. 103888; doi: 10.1016/j.aei.2025.103888.

[10] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, "Leveraging LLM agents and digital twins for fault handling in process plants," in *Proc. IEEE ETFA*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205597.

[11] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025. [Online]. Available: https://arxiv.org/abs/2510.11300v2

[12] C. Graul, W. Żarski, D. Mikołajewski, and I. Rojek, "Ai agents in industry 4.0: AAS–OPC UA–LLM architecture as the foundation of intelligent manufacturing systems in the context of industrial enterprise implementation," *Applied Sciences*, vol. 16, no. 11, p. 5428, 2026, doi: 10.3390/app16115428.

[13] D. Schall, "Safe integration of large language models into industrial process control: A multi-agent architecture with P&ID-grounded validation," *Autonomous Intelligent Systems*, vol. 6, 2026, article no. 14; doi: 10.1007/s43684-026-00136-1.

[14] S.-C. Lin and C.-Y. Chen, "An integrated framework featuring policy-governed agentic AI for closed-loop manufacturing control with multi-source sensor–MES–ERP," *The International Journal of Advanced Manufacturing Technology*, vol. 143, pp. 6405–6428, 2026, doi: 10.1007/s00170-026-17806-2.

[15] Y. Xia, N. Jazdi, and M. Weyrich, "An architecture for integrating large language models with digital twins and automation systems," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205636.

[16] L. M. Vieira da Silva, A. Köcher, and F. Gehlhoff, "Beyond formal semantics for capabilities and skills: Model context protocol in manufacturing," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–4, doi: 10.1109/ETFA65518.2025.11205601.

[17] K. Köhle, D. Anicic, T. A. Runkler, K. Dorofeev, A. S. Thuluva, and R. Graf, "Virtually commissioned semantic recipes based on OPC UA skills," in *Proc. IEEE 8th Int. Conf. on Industrial Cyber-Physical Systems (ICPS)*, 2025, pp. 1–6, doi: 10.1109/ICPS65515.2025.11087832.

[18] X. Jiang *et al.*, "Agentic AI for safety-aware process monitoring and fault diagnosis: A review," *Processes*, vol. 14, no. 13, 2026, art. no. 2112; doi: 10.3390/pr14132112.

[19] Model Context Protocol contributors, "Model Context Protocol: Tools," Specification, 2026, accessed: Oct. 20, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/server/tools

[20] ISA, "ISA-88.00.01-2010: Batch control—part 1: Models and terminology," International Standard, 2010.

[21] ——, "ANSI/ISA-95.00.01-2025 (IEC 62264-1 mod): Enterprise-control system integration—part 1: Models and terminology," International Standard, 2025.

[22] J. Moyne, E. del Castillo, and A. M. Hurwitz, Eds., *Run-to-Run Control in Semiconductor Manufacturing*, 1st ed. Boca Raton, FL: CRC Press, 2000.

[23] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern., Part A*, vol. 30, no. 3, pp. 286–297, 2000, doi: 10.1109/3468.844354.

[24] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM 48th Int. Conf. on Software Engineering (ICSE)*, 2026, pp. 2938–2950, doi: 10.1145/3744916.3764546.

[25] E. Debenedetti *et al.*, "Defeating prompt injections by design," 2025, arXiv preprint arXiv:2503.18813. [Online]. Available: https://arxiv.org/abs/2503.18813

[26] R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Hum. Factors*, vol. 52, no. 3, pp. 381–410, 2010, doi: 10.1177/0018720810376055.

[27] L. Bainbridge, "Ironies of automation," *Automatica*, vol. 19, no. 6, pp. 775–779, 1983, doi: 10.1016/0005-1098(83)90046-8.

[28] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90, doi: 10.1145/3605764.3623985.

[29] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1831–1847. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei

[30] IEC, "IEC 62443-3-3:2013: Industrial communication networks—network and system security—part 3-3: System security requirements and security levels," Standard, 2013.

[31] ——, "IEC 61511-1:2016: Functional safety—safety instrumented systems for the process industry sector," International Standard, 2016.